PROPOSAL · ONE MODEL, MANY RADARS · RAW ADC · CUBELEARN

Scene-Space CubeLearn

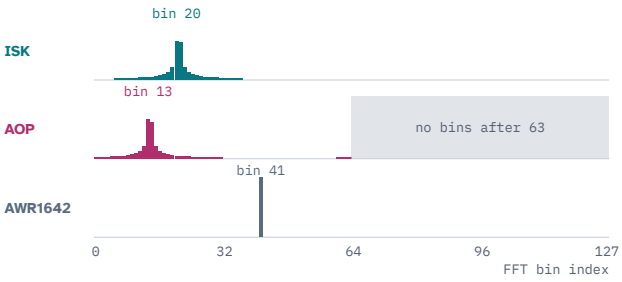
Every number in an ADC file has a physical address: the frequency the radar was sending, the moment it measured, and the pair of antennas that measured it. The radar description gives us this address. We use it to place every number on one shared map of the scene, in metres, metres per second and directions. CubeLearn then learns on the map instead of on tensor positions. One set of weights can serve every radar. A new radar needs only its description.

The problem

On the fast-time axis, tensor position 20 means about 2.0 m for the CubeLearn board and 3.0 m for the mmRadPose board. So the same reflector lands at different positions, and one position means different things. A shared network has to learn every pattern twice. On a map in metres, the reflector sits in the same place for every radar. Only the sharpness differs.

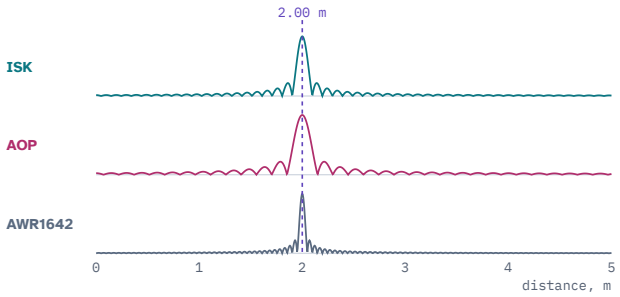
Move the reflector 2.00 m

What an FFT gives: a tensor position



ISK: bin 20 of 256 · AOP: bin 13 of 64 · AWR1642: bin 41 of 512

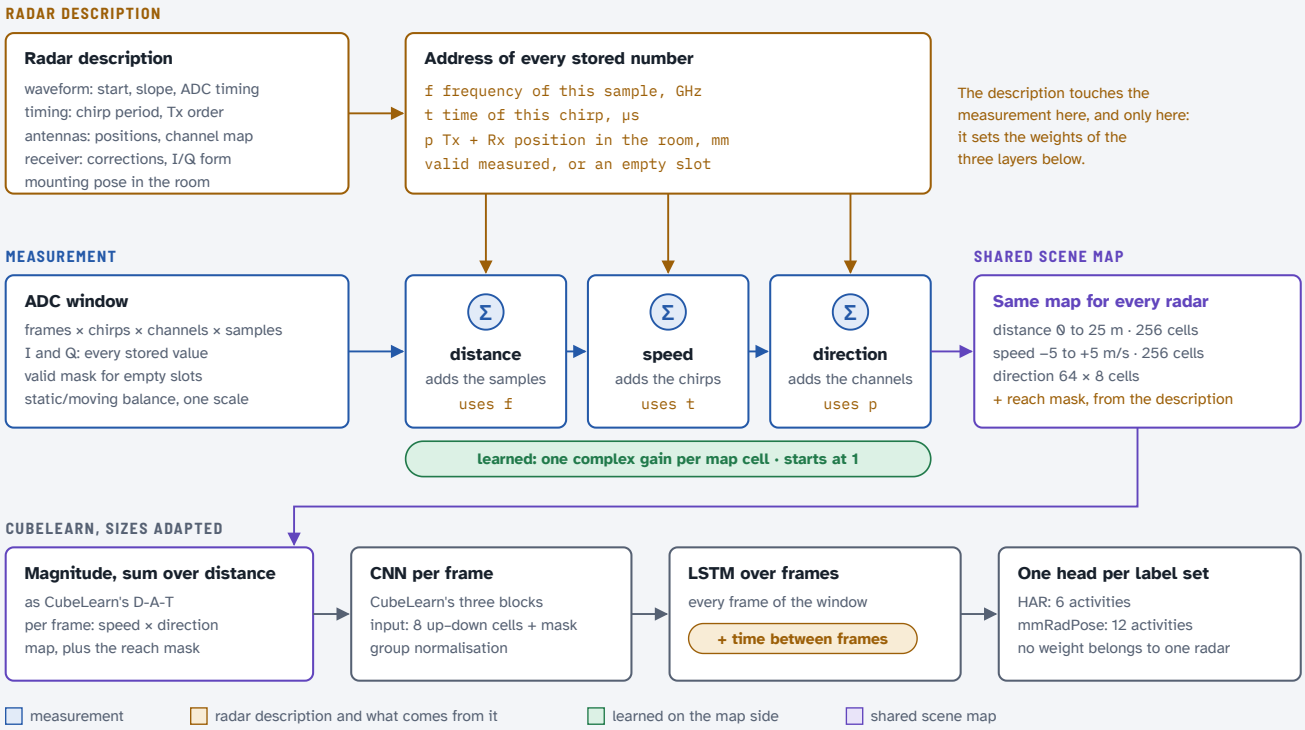
What the scene layers give: a place on the shared map



All three at 2.00 m. Sharpness: 9.8 cm, 14.8 cm, 4.9 cm.

Real settings: IWR6843ISK as in CubeLearn HAR (256 samples at 10 MS/s), IWR6843AOP as in mmRadPose (64 samples at 3.8 MS/s), and AWR1642 at 77 GHz with the People Walking settings, an example of a radar we never train on. Distance only; speed works the same way with chirp times, direction with antenna positions.

One window through the model



The radar description meets the measurement in three places only: the three layers that build the map. The front end learns only on the map side. After the map, the network is CubeLearn with adapted input sizes.

What changes in CubeLearn:

1. The three complex layers are built from addresses, and all write onto one shared map.
2. Their learned part becomes one gain per map cell.
3. A reach mask joins the CNN input, and the time between frames joins the LSTM input.

Two fixes from the earlier runs stay: group normalisation, and scaling the moving part of the signal separately from the static part.

The three layers

Think of many microphones in a room. To listen to one spot, shift each recording by its delay from that spot, then add them up. Sound from that spot adds up. Everything else cancels.

The radar version: for each map cell, physics says how a small reflector in that cell would turn the phase of each stored number. The layer turns each number back by that amount and adds. A cell with a real reflector gets a large sum; empty cells cancel. For one radar on its own grid, this is the

FFT that CubeLearn starts from, plus the timing fix below. Written with addresses instead of tensor positions, it works for any radar.

LAYER	ADDS UP	USES	SHARED AXIS
distance	the samples of one chirp	`f`, frequency of each sample	0 to 25 m, 256 cells of 9.8 cm
speed	the chirps of one frame	`t`, time of each chirp	−5 to +5 m/s, 256 cells of 3.9 cm/s
direction	the channels	`p`, Tx + Rx position	64 left–right × 8 up–down

Two things come for free. The three transmitters fire one after another, so a moving arm changes phase between them, and a plain FFT reads that change as a change of direction. The time `t` includes each transmitter's own firing time, so the speed layer removes this error by itself. And positions are in room coordinates, so a board turned on its side still paints the same scene.

> In general, the description answers one question for every stored number and every map cell: what would this number read if one small reflector sat in this cell? Antenna patterns, filter responses, phase codes and beam weights all fit into that answer. For the two boards here, the answer is a plain phase.

Where learning lives

CubeLearn learns its three FFT matrices. Each learned weight belongs to a tensor position, so it cannot be shared across radars.

We keep the learning but move it onto the map. Each map cell gets one learned complex gain, starting at 1. For example, the model can learn to turn down zero speed, where walls and furniture sit, or to turn up 1 to 4 m, where people stand. A map cell means the same thing for every radar, so one gain serves all of them, including radars we have not seen.

We do not learn weights as functions of absolute frequency, time or antenna position. Such weights can only be trusted for values seen in training, and a 77 GHz radar would fall outside them.

Is one gain per cell enough? CubeLearn's matrices are freer. So we test three levels of freedom, all on the map side: none (pure physics, like CubeLearn's fixed-FFT variant), one gain per cell (proposed), and a full learned matrix per axis. The full matrix can express anything CubeLearn learns on one radar. If it wins clearly, we take it; it is still shared by all radars.

Where the rest of the description goes

The aim is to keep what is needed to read a measurement, not to push every field into every layer.

PART OF THE DESCRIPTION	WHAT IT BECOMES
Tensor structure: axes, sizes, active channels, empty slots, padding, missing or saturated readings	The address of each number and a valid flag. Invalid slots add nothing; a measured zero adds zero. No size is fixed anywhere.
Antennas: positions, enabled elements, stored channel to Tx/Rx pair, mounting pose	The position `p` of each channel, in room coordinates. Known element patterns, polarisation or beam weights replace the plain phase with the known response.
Waveform: start, slope or sweep shape, the part the ADC saw, phase codes	The frequency `f` of each sample, taken from the part of the sweep the ADC actually saw. Phase codes are undone when reading.
Sampling and timing: rate, ADC delay, Tx schedule, loops, frames, gaps	`f` and `t`. Settings that change between chirps or frames need no special case, because each number has its own address. The time between frames goes to the LSTM.
Receiver: gains, filters, channel calibration, I/Q imbalance, noise	Per-channel corrections when reading. Known filter responses join the distance weights. Overall gain is removed by the scaling. Noise level is not used today; it would join the beam later.
Representation: real or complex, I/Q order and sign, units, earlier processing	One standard complex form when reading. A real-only radar gives real numbers and a shorter reach; we never invent Q. Corrections already applied are recorded so they are not applied twice.
Several sensors: relative pose, clocks, phase coherence	Their numbers get addresses in the same room and on the same clock. Sensors that are not phase-locked are added on the map, not inside the sums.

Symmetries

Must not change the answer. Each item is a unit test on the untrained model, exact to float error.

- Renaming channels, chirps or samples together with their addresses. Every layer adds over addresses. No weight belongs to a tensor position, and there are no learned antenna IDs.

- Storage layout and padding. Masked slots add nothing.
- One complex factor on all numbers: overall phase or gain. The magnitude and the scaling remove it.
- The choice of array origin or time origin. Positions are measured from the array centre, times from the middle of the frame.
- Turning the board, when its mounting pose turns with it.

Must move the answer by the same physical amount. A person who stands farther, moves faster or steps to the side shifts the map by exactly that much, on every radar. So one 3 × 3 convolution covers the same patch of the world on every radar. On tensor positions it does not.

May change the answer. A new waveform, timing or array. It changes the addresses and acts only through them.

What the data showed

I looked at a few recordings from each dataset before trusting any address. Four facts matter.

> **The AOP phase flip is already corrected.** The board guide documents a 180° flip on two receivers. In mmRadPose, at the person's distance, all 12 channels have smoothly changing phases (0 to 68°) with no 180° jumps. Applying the flip again would be wrong.

> **"Q is four times I" is a fixed pattern.** In mmRadPose the extra Q energy repeats identically in every chirp, so it sits at zero speed. The moving part is balanced (Q/I about 1.1, the same as HAR). This pattern is a main reason only about 0.1% of mmRadPose energy moves.

> **Both datasets show a faint mirror copy of the person.** It is about 11 dB weaker in HAR and 12 to 20 dB weaker in mmRadPose. This is I/Q imbalance in the receiver. On the map the copy appears at the opposite speed and the mirrored direction.

> **HAR receivers are not phase-calibrated.** The phase pattern across the four receivers repeats for both transmitters and is not a straight line. This points to receiver phase offsets of roughly 20 to 30°.

So the descriptions of these two radars are incomplete. The last two facts are receiver corrections that the releases do not give. The second fact also explains why one overall scale fails: it would make a moving arm about five times fainter on mmRadPose than on HAR. That is why the moving part keeps its own scale.

Every stored value reaches the model

STEP	CUBELEARN HAR · ISK	MMRADPOSE · AOP
original file	float64 (2, 20, 128, 12, 256): I/Q, frames, chirps, channels, samples	complex128 (T, 4, 4, 64, 128): frames, grid row, grid column, samples, chirps; T from 75 to 535
cache	int16 with all 12 channels. Today's cache keeps 8, so we rebuild it, checking that every value converts exactly.	int16 (T, 12, 128, 64, 2). The 4 empty grid slots are checked to be zero in every file, then kept as structure, not data.
loader	one window = the whole 20-frame recording	30-frame windows back to back; the last, shorter window is kept with a frame mask
model input	every I and Q value of every frame, chirp, channel and sample enters the distance layer; nothing is cropped or resampled	
after the layers	per frame: 256 speeds × 64 × 8 directions, summed over distance, plus the reach mask; mmRadPose distance cells beyond 9.5 m are masked	

How we test it

Same person split as the earlier runs. HAR: people 0–4 train, 5 validation, 6–7 test. mmRadPose: people 1, 2, 4–8 and 10 train, 11–12 validation, 3 and 9 test. These test people were already used once, by the cropped runs, and we will say so. Two test people per dataset is a weak test; for stronger evidence we can rotate the test people over folds.

Every run: three or more seeds; accuracy and macro-F1 per recording, per dataset and per test person.

1. **CubeLearn alone** on each dataset, all inputs. The baseline. Its matrices start from each board's own FFT and antenna steering.
2. **Our model alone** on each dataset. Only the front end differs. This checks "no degradation".
3. **Our model on both datasets**, one head per label set. The main question.
4. **CubeLearn matrices on both datasets**, padded. Shows what the shared map adds.

The joint run sees the same draws from each dataset as the single runs. One more single run gets as many updates as the joint run, to separate more data from more steps. Validation picks every setting.

Ablations, each answering one question:

- How much learning on the map side? None, one gain per cell, or a full matrix.
- Does correct timing matter? Remove the transmitter delays from `t`.
- Does the frame interval matter? Remove it.
- Does the model read the description? Shuffle the channel-to-position map at test time. Accuracy must fall.

Later: a radar we never trained on

After the map, radars differ in only two ways: how sharp they see and how far. Physics gives both. Send one imaginary point reflector through the radar's own layers; the blurred spot that comes out is the radar's **beam**. Radio astronomers always publish an image together with its beam. We would give the CNN the beam too, through feature scale-and-shift.

With two radars, the beam is just a dataset name. But any subset of a radar's numbers is a real measurement by a smaller radar. Fewer samples mean less bandwidth, fewer chirps a shorter look, every second chirp a lower repetition rate, fewer channels a smaller array, and their addresses are exact. So two real radars give thousands of exact training radars, and the description stops being a dataset name. Test inputs always keep every value. This is the next step, not today's.

Risks

- Two radars from one chip family, both 60 GHz, both 3 Tx × 4 Rx. Today's test cannot show transfer to a new radar.
- Some settings are documented defaults, not recorded values: the 6 µs ADC delay, the 160 and 83.3 µs transmitter spacing, the AOP start frequency and slope. An error stretches or shifts the map a little.
- Missing receiver corrections leave faint mirror copies and a slight blur.
- The usual approximations: far field, three separate layers, one centre frequency for speed and direction. CubeLearn and standard FFT processing make the same ones.
- The shared map is about 16 times more front-end work than CubeLearn's own grid. That is fine on H100s, frame by frame, and it is not a reason to crop.

Decisions for us

Receiver corrections

Estimate the I/Q and receiver-phase corrections from training recordings, without labels: a few numbers per receiver. Or leave them out and accept the blur.

Recommend: estimate. They are facts about the radar and belong in its description.

Freedom on the map side

Main run with one gain per cell. Test none and a full matrix beside it.

Recommend: this order. Simplest first; more only if it wins on validation.

The shared map

Fixed: 0 to 25 m, ±5 m/s, 64 × 8 directions, as fine as the sharpest radar we use. A sharper radar later needs a finer map.

Recommend: fixed for now.



THE UNIVERSITY of EDINBURGH

Edinburgh Research Explorer

CubeLearn

End-to-end Learning for Human Motion Recognition from Raw mmWave Radar Signals

Citation for published version:
Zhao, P, Lu, CX, Wang, B, Trigoni, N & Markham, A 2023, 'CubeLearn: End-to-end Learning for Human Motion Recognition from Raw mmWave Radar Signals', *IEEE Internet of Things Journal*, vol. 10, no. 12, pp. 10236-10249. https://doi.org/10.1109/JIOT.2023.3237494

Digital Object Identifier (DOI):
10.1109/JIOT.2023.3237494

Link:
Link to publication record in Edinburgh Research Explorer

Document Version:
Peer reviewed version

Published In:
IEEE Internet of Things Journal

General rights
Copyright for the publications made accessible via the Edinburgh Research Explorer is retained by the author(s) and / or other copyright owners and it is a condition of accessing these publications that users recognise and abide by the legal requirements associated with these rights.

Take down policy
The University of Edinburgh has made every reasonable effort to ensure that Edinburgh Research Explorer content complies with UK legislation. If you believe that the public display of this file breaches copyright please contact openaccess@ed.ac.uk providing details, and we will remove access to the work immediately and investigate your claim.



Download date: 02. Nov. 2026

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

CubeLearn: End-to-end Learning for Human Motion Recognition from Raw mmWave Radar Signals

Peijun Zhao†, Chris Xiaoxuan Lu‡, Bing Wang⋆, Niki Trigoni†, and Andrew Markham†

***Abstract*—mmWave FMCW radar has attracted huge amount of research interest for human-centered applications in recent years, such as human gesture and activity recognition. Most existing pipelines are built upon conventional Discrete Fourier Transform (DFT) pre-processing and deep neural network classifier hybrid methods, with a majority of previous works focusing on designing the downstream classifier to improve overall accuracy. In this work, we take a step back and look at the pre-processing module. To avoid the drawbacks of conventional DFT pre-processing, we propose a complex-weighted learnable pre-processing module, named CubeLearn, to directly extract features from raw radar signal and build an end-to-end deep neural network for mmWave FMCW radar motion recognition applications. Extensive experiments show that our CubeLearn module consistently improves the classification accuracies of different pipelines, especially benefiting those simpler models, which are more likely to be used on edge devices due to their computational efficiency. We provide ablation studies on initialization methods and structure of the proposed module, as well as an evaluation of the running time on PC and edge devices. This work also serves as a comparison of different approaches towards data cube slicing. Through our task agnostic design, we propose a first step towards a generic end-to-end solution for radar recognition problems.**

***Index Terms*—mmWave radar, end-to-end neural network, motion recognition**

I. Introduction

mmWave FMCW radar was mainly used on high-end cars and military vehicles many years ago, due to its bulky size and high cost. With the recent development in low cost single-chip mmWave radar, (e.g., TI mmWave sensors), more and more possibilities have been explored in indoor applications, such as vital sign monitoring [1], gesture recognition [2], [3], fall detection [4], [5], and human activity recognition (HAR) [6]–[8]. As mmWave radars are less obtrusive than cameras, they have significant potential for domestic applications, where concerns about privacy dominate.

Typical methods for FMCW radar gesture or activity recognition follow a two-stage pipeline: radar signal pre-processing, typically with Discrete Fourier Transform (DFT), and a data-driven classifier for task-specific recognition purposes. In the classical radar processing chain, there are a number of different levels of data representations, ranging from Range Profile signatures, Range-Doppler and Range-Angle maps [1] , micro-Doppler signature, to point clouds. For gesture recognition and activity recognition tasks, maps and point clouds are the most widely adopted data representations [9], [10]. Point cloud generation relies on hand crafted parameters, and generating point clouds requires receiver array which is not available in some radar configurations (e.g., SISO radar). Besides, point cloud can hardly capture find-grained movements like finger gestures, which makes it a less universal approach. As a result, most previous works are based on maps + neural network classifier hybrid pipelines.

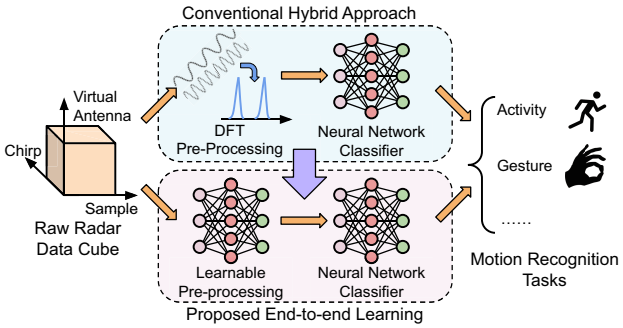


Fig. 1. We propose CubeLearn, a learnable pre-processing module to replace conventional DFT and build end-to-end deep neural networks for mmWave FMCW radar motion recognition tasks.

DFT serves as a very efficient way to extract Range, Doppler and Angle-of-Arrival (AoA) information from the raw radar signal to generate different maps. However, it also has limitations like using non-adaptive basis and resolution limitations. To overcome such limitations, researchers have been trying to use end-to-end deep neural networks for radar applications in recent years. The difficulty here lies in that the raw radar ADC samples are complex values corresponding to the downmixed baseband, and both the magnitude and the phase are important. In previous works on end-to-end learning for radar applications, either only the real part is used [11], [12], which loses some information; or the real part and the imaginary part are treated as two channels [13], which loses the physical meaning, and using a specific network structure could make it less universal.

In this work, we explore replacing DFT pre-processing with a complex-weighted learnable pre-processing module named CubeLearn to improve motion classification accuracy, which is evaluated on human gesture and activity recognition tasks. To the authors' best knowledge, this is the first work to use stacked complex linear layers to replace DFT pre-processing for FMCW radar pipelines, and the first in-depth evaluation of a purely end-to-end radar network. Our intuition is that by

†Department of Computer Science, University of Oxford, United Kingdom (firstname.lastname@cs.ox.ac.uk)

‡Department of Informatics, University of Edinburgh, United Kingdom (xiaoxuan.lu@ed.ac.uk, Corresponding Author)

⋆Department of Aeronautical and Aviation Engineering, Hong Kong Polytechnic University. This work was finished when he was with University of Oxford.

[1]We use the term 'maps' to refer to 2D and 3D representations, which will be introduced in Section IV in detail.

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

making the entire network learnable and data-driven, we can allow the network to better focus on important radar features, leading to higher overall accuracy. In particular, significant performance improvements are observed for those simpler models, indicating that our proposed CubeLearn module could be especially helpful for low-end radars (e.g., with single Tx/Rx, or limited memory/computational ability) to achieve recognition performance close to a more sophisticated radar. Besides, compared to previous end-to-end works where the proposed methods are designed for specific applications, our work proposes a more generic method for different radar recognition tasks.

Our contributions in this work are listed as follows:

- We propose CubeLearn, a learnable pre-processing module based on complex neural networks to replace the conventional DFT pre-processing, allowing the network to focus on task-specific radar signatures.
- Through extensive evaluation, we show that our proposed method can increase the accuracy of different gesture/activity recognition pipelines and accelerate the training process.
- We also provide a detailed, in-depth exploration of the impact of different pre-processing and classifier combinations, in terms of task accuracy and computational load on both PC and Raspberry Pi devices.
- We release the code of the proposed CubeLearn module and our own collected dataset to encourage further research on this topic [2].

In the rest of the paper, we first discuss related work, mmWave radar background and commonly seen pipelines. Then we present our proposed CubeLearn module in Section V. In Section VI we introduce the dataset collection and experiment configuration, followed by evaluation in Section VII. In the remaining sections we provide extensive ablation study and complexity analysis, and a discussion on model selection, robustness and possible ways to utilize the elevation data as well. Section XII finally concludes the paper.

II. Related Work

A. mmWave Radar Map Based Human Gesture/Activity Recognition

Pre-processing		Classifier	Related Works
1D Profile	R + A + D	LSTM	[14]
2D Map	D-T	2DCNN	[7], [15]–[18]
	R-D	2DCNN-LSTM	[3], [19]
	R-D	2DCNN-TCN	[20]
	R-D	ANN	[21]
	R-D + R-A	2DCNN-LSTM	[22]
	R-T + D-T	2DCNN	[23]
	R-T + A-T	2DCNN	[24]
3D Map	R-D-T	3DCNN	[25], [26]
	R-D-A	2DCNN + LSTM	[27]

TABLE I
MAP BASED MMWAVE FMCW RADAR GESTURE AND ACTIVITY RECOGNITION. R: RANGE; A: ANGLE; D: DOPPLER; T: TIME.

mmWave radar based human gesture/activity recognition is one of the most studied areas in mmWave sensing. Mainstream recognition methods are based on 1D/2D/3D data representations. Recent works on mmWave radar based human gesture/activity recognition are summarized in TABLE I. As introduced in Section III, conventional DFT pre-processing is used to extract Range (R), Doppler (D) and AoA (A) information from the raw radar data cube. In most cases the Range-Doppler Map, Range-Angle Map or micro-Doppler signature is directly used as neural network input [7], [15]–[20], [22]–[26], as they are fixed-sized and can be processed similarly to images. Other works manually extract features and feed them into the downstream classifier [14], [21], [27].

There are many types of 2D and 3D data representations, such as the Range-Doppler Map used in [3], [19]–[22]. To form a range-Doppler map, we need to transmit multiple chirps, and the duration is called 'Coherent Processing Interval (CPI)'. The CPI dimension (T) can be used as a separate dimension for the neural network to extract temporal information [3], [19], [20], [22], [27], or use together with other dimensions to be processed as part of the data map [15], [16], [25], [26]. In our work we consider different map based pipelines in previous works, together with other possible combinations, as our baselines.

B. End-to-end mmWave Radar Gesture and Activity Recognition Methods

Researchers have also been trying to build end-to-end neural networks for radar applications. The main difficulty of designing an end-to-end structure is to handle the complex input. The magnitude and phase information are both important in the raw IF signal, as we need to extract the distance, velocity and AoA information from the frequency and phase of the raw signal.

Sakamoto et al. proposed converting the I/Q data received by CW radar receiver into an image for hand gesture recognition with convolutional neural networks [28]. Complex valued convolutional neural networks are also proposed to extract information from radar micro-Doppler signatures for activity recognition [29]. Zhao et al. proposed treating the real and imaginary part as two separate channels which is processed similar to images with real-valued convolutional layers for human activity classification [13]. There are also some works that use real-valued input. Stadelmayer and Santra proposed using a parametric convolutional neural network (2D Sinc Filter and 2D Wavelet Filter) [11] to extract Range and Doppler information from raw radar data. Ye et al. proposed using two real-valued convolutional layers with Fourier initialization for human-activity classification [12], [30] on Continuous-wave radar data. However, their methods lose half of the information from the imaginary part.

To the authors best knowledge, this work is the first to use stacked complex linear layers to replace the conventional DFT pre-processing to directly extract information from raw mmWave FMCW radar data, and to build end-to-end deep neural network together with downstream deep classifier which directly takes raw radar complex data cube as input for recognition tasks.

III. mmWave FMCW Radar Background

In a typical FMCW radar configuration, a CPI consists of multiple chirps, which are short periods of signals whose fre-

[2]https://github.com/zhaoymn/cubelearn

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

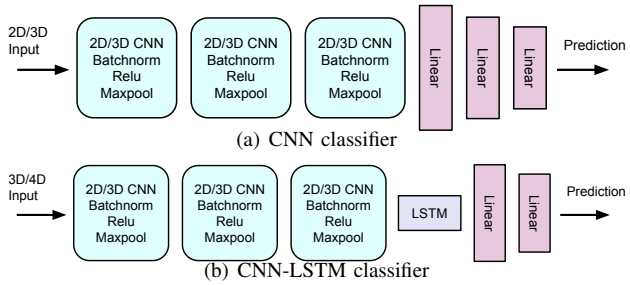
quency increases linearly with time. We use Texas Instruments IWR6843 mmWave radar in this work, which features a 3 transmitter, 4 receiver MIMO antenna array, but we note that our architecture is easily generalizable to different antenna configurations. For each chirp, typically a single transmitter is activated (with the TDM-MIMO [31] principle), and the received signals at each receiver antenna are mixed with the transmitting signal followed by low-pass filters to produce an intermediate frequency (IF) signal that is further sampled by analog-digital converter (ADC). For each CPI, mmWave radar generates a raw data cube, with the three axes namely corresponding to: samples in a chirp, chirp loops, and Tx-Rx pairs.

In conventional processing, DFT is used to extract range, Doppler and AoA information along the 3 dimensions of the raw radar data cube, respectively. As the transmitting frequency increases linearly with time during a chirp, the reflection from a target at a certain distance introduces a corresponding frequency component in the IF signal, which equals to the round trip time multiplied by the frequency slope of the chirp. We can estimate the distance to different targets by extracting different frequency components with DFT, which produces a 'Range Profile'. The phase of a peak corresponding to a certain target in the Range profile changes across chirps if the target is moving, due to the slight round-trip distance change, and we can apply another DFT across chirps to extract the phase variation in order to infer the radial velocity of the object, which is often called 'Doppler DFT'. Furthermore, the AoA of the target introduces phase differences at the antennas in the receiver array because of the slight difference in the total length the signal travels. This can also be estimated with DFT, which is called 'Angle DFT'. Transforming the information from the sampled time/space domain to the frequency domain is the key for extracting range, Doppler and AoA information in FMCW radar data processing.

IV. Gesture and activity recognition models

As discussed in the previous section, the mmWave radar is able to produce a raw data cube for each CPI, with 3 axes as sample, chirp and virtual antenna, respectively, and range, Doppler and AoA information can be extracted from these three dimensions. To represent the information on these three dimensions. We often adopt maps as a straightforward data representation. Peaks in the maps correspond to detected targets. For example, a peak in the 2D Range-Doppler maps represents a target at a certain distance with a certain radial velocity. Since in this work we are studying gesture/activity recognition, which typically lasts several CPIs, we have an additional CPI dimension. As a result, with different combinations of information from three dimensions in the data cube, together with the CPI dimension, we supply 2D, 3D or 4D data as input to the downstream neural network classifier.

In this study, we evaluate two types of commonly adopted neural network classifiers: CNN and CNN-LSTM. The architecture of the two types of classifiers is shown in Fig. 2. We use three convolutional layers, each followed by batch normalization, activation (ReLU), and max-pooling. The output of the convolutional layers is flattened, followed by either LSTM and two fully connected layers, or three fully connected layers, to produce the prediction.



(a) CNN classifier

(b) CNN-LSTM classifier

Fig. 2. Two types of downstream classifier architectures we use in this work.

Data Cube Slicing	Pre-processing	Classifier
first Tx-Rx pair and first chirp	R-T	2DCNN
first Tx-Rx pair, range aggregation	D-T	2DCNN
first chirp, range aggregation	A-T	2DCNN
first Tx-Rx pair	R-D-T	3DCNN/2DCNN-LSTM
first chirp	R-A-T	3DCNN/2DCNN-LSTM
whole cube, range aggregation	D-A-T	3DCNN/2DCNN-LSTM
whole cube	R-D-A-T	3DCNN-LSTM

TABLE II
MAP BASED MMWAVE FMCW RADAR GESTURE AND ACTIVITY RECOGNITION.

The CPI can sometimes be used as one dimension in the map, e.g., in a Range-Doppler-Time Cube, which can be fed into 3DCNN classifier, or used as sequence timestamps, to act as input into LSTM. We use TABLE II to summarize possible data slicing, pre-processing and downstream classifier combinations. Note that since we are using FMCW radar, for Doppler-Time, Angle-Time and Doppler-Angle-Time pre-processing, we still extract Range information first, and then estimate Doppler and/or AoA information for each Range bin, then sum along the Range axis. Some of the combinations have appeared in previous works, as we discuss in Section II, and in these works the selection of the pipeline is largely based on the target application and radar configuration.

V. Learnable pre-processing

A. Problems with Conventional DFT pre-processing

For raw radar signals, the information of the targets is not encoded in certain data points, but rather hidden in the phase and magnitude variation of the raw signal. For example, the distances to the targets are encoded in the frequency components of a single chirp, and the Doppler and AoA information are hidden in the phase variation, either from time domain (chirps) or spatial domain (receiver array).

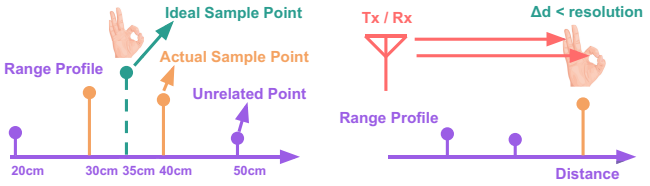
To expose such information, the pre-processing is often performed with conventional signal processing techniques, typically through DFT, as discussed before. The DFT can be viewed as a linear transformation of the raw radar data from the time/spatial domain to the frequency domain. Linear transformation can be viewed as change of basis. In DFT, the data is represented with equally spaced Fourier bases in the frequency domain, and the information corresponding to the targets is directly exposed. The pre-processed output can be largely regarded as 2D/3D images to be fed into downstream neural networks based on architectures widely explored in the

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

computer vision area. Note that though DFT is a reversible operation, in previous pipelines the phase information of the transformed data is lost before feeding into the downstream classifier because of the modulus operation.

Though conventional DFT pre-processing is simple, efficient and straightforward, there are two issues which impact its utility:



(a) Fixed Fourier bases cannot best capture target information, and carry information not related to the target.

(b) Resolution limitation makes it hard to capture find-grained movements.

Fig. 3. Limitations of the conventional DFT pre-processing.

Lack of adaptive transformation bases. The conventional DFT is a universal solution for extracting frequency domain information, and is not designed specifically for certain tasks. In the DFT, all the Fourier transformation bases are equally spaced and fixed, so the transformed data points might not best represent the target, as illustrated in Fig. 3(a). Note also the redundancy, as the part of the conventional DFT output that we are actually interested in only occupies a fraction of the whole pre-processing output. The non-informative part of the output may contain multipath reflections, noise and environmental reflections. Sending this information into the downstream neural network will lead to a higher computational cost, and even worse, can make the neural network learn fake features and overfit to the training data. It is possible to have a specific DFT for each task, with different FFT lengths, non-uniform frequency spacings or frequency masks such that the domain transformation can focus more on relevant features corresponding to specific tasks. However, this means that we must choose specific parameters for each task, which relies on the experience of the developer and it is very time-consuming to tune these hyper-parameters.

Limited resolution. Another problem is that while the DFT is complete and invertible, it has an inherent resolution limitation, which makes it hard for the DFT transformation output to represent the target properties on certain tasks, as demonstrated in Fig. 3(b). For example, the Range resolution of mmWave radar is typically $>5cm$, which makes it difficult to distinguish between two fingers in hand gesture recognition. Also, the resolution limitations of Doppler and AoA make it hard to capture fine-grained movements, e.g., movements of two adjacent fingers. Note that such problem could not be simply fixed by 'Zero Padding' in the time domain, as 'Zero Padding' in time domain corresponds to interpolation in frequency domain, which does not increase the resolution. This is because the resolution is a function of the raw signal bandwidth. Generally speaking, the resolution limitation is inherent to the transmit waveform. However, with additional prior knowledge, we are able to achieve better resolution i.e. super-resolution techniques. For example, with the number of signal sources as a prior knowledge, Multiple Signal Classification (MUSIC) algorithm can identify two targets that are not separable with DFT processing [32]. In our case, the neural network model is trained with training data, which inherently carries rich prior knowledge. As a result, with additional information, it can potentially overcome the resolution limitation of DFT based processing.

B. CubeLearn Module Architecture

To deal with the above-mentioned limitations of conventional DFT pre-processing, in this work we present CubeLearn module to replace the DFT pre-processing, which can be trained together with the downstream classifier in an end-to-end way, and aim to learn the best domain transformation for a specific task in a data-driven manner.

A neural network does not suffer from the limitations of conventional DFT. Through end-to-end training, the network is able to learn the transformation bases that best suit the target task. In addition, with prior knowledge acquired from training data, the neural network could be able to achieve better resolution. In fact, recent research has shown that super-resolution of Range, Doppler and AoA can be achieved through deep neural networks for raw radar signals [33], [34]. Furthermore, the pre-processing module could be trained to extract more informative features, rather than noise and environmental reflections, potentially carrying more useful information to the downstream classifier.

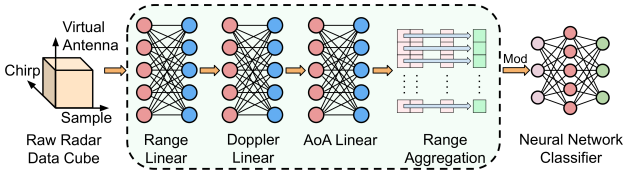


Fig. 4. CubeLearn Architecture.

The architecture of our proposed CubeLearn module is shown in Fig. 4. The basic idea behind the design is to follow the conventional DFT processing method, as introduced in Sec. III, but in a learnable way. As the raw radar signal is complex-valued, we use complex linear layers to directly consume the raw radar signal as input. Instead of processing all the information in the raw radar data cube in one layer, we use a stacked hierarchy of complex linear layers, each layer replacing a certain step in conventional DFT pre-processing (i.e., Range, Doppler and AoA). This is because if we are to use a single complex layer, there would be too many parameters, making the neural network significantly more complex.

As we are using FMCW radar, the first step is to extract Range information from each chirp with a complex linear layer. This can be followed by other complex linear layers to extract Dopper and AoA information for different ranges. We can also aggregate the Doppler and AoA information at different ranges by calculating the sum along the Range axis. In conventional DFT pre-processing, the order of performing DFT on different dimensions of the input raw data cube does

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

not affect the output. As a result, though we are following the conventional DFT pre-processing to organize the complex linear layers, the order of the linear layers could also potentially be swapped, which could be studied in future work.

To make the proposed CubeLearn module fairly act as a drop-in for the conventional DFT, we calculate the absolute (modulus) value before feeding the processed data into the downstream neural network classifier.

C. CubeLearn Module Weight Initialization

The DFT is a linear transformation from the time domain to the frequency domain. This can be represented by a complex linear layer with no bias in a neural network. We use DFT pre-processing as prior knowledge to initialize our proposed CubeLearn module, and let the neural network adjust the weights to adapt to a specific task through end-to-end training. The complex linear layer output can be represented as:

$$output[k] = \sum_{m=0}^{M-1} w(k,m) input[m] \tag{1}$$

and we initialize the weights as

$$w(a,b) = e^{-j\frac{2\pi}{N}ab} \tag{2}$$

which exactly represents DFT. The bias of the complex linear layers are set to 0.

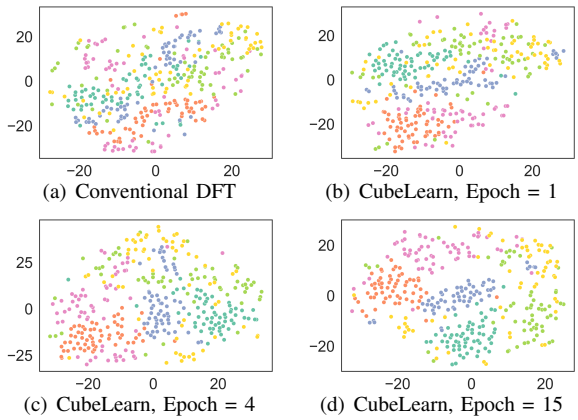
For Doppler and AoA transformation layers we also need to shift the upper half and the lower half of the weights, since we want zero speed and zero AoA (i.e. in front of the sensor) initially to be in the middle of the learned map.

Note that though we initialize the complex layers as DFT, through end-to-end training the transformation is very likely to no longer be Fourier Transform once trained.

D. Effect of CubeLearn Module

To better show the effects of our proposed CubeLearn module compared to the conventional DFT pre-processing and validate our assumptions, in this part we demonstrate how the proposed CubeLearn module adaptively adjust the weights, by visualizing the output of both the conventional DFT pre-processing module and the CubeLearn module through the training process. We use t-SNE, a widely adopted method for dimension reduction and data visualization [35]. Without loss of generality, we use D-T pre-processing + 2DCNN classifier model on Hand Gesture Recognition task here as an example. Detailed definition of the task as well as the description of the dataset is introduced later in Section VI. There are 12 hand gestures in total, however, to make the figures more clear, we only visualize the pre-processed output of 6 gestures.

In Fig. 5, we first visualize the output of the conventional DFT pre-processing in Fig. 5(a), and then the change of the output of the proposed learnable D-T pre-processing module in Fig. 5(b)-(d). From the Fig. 5(a) we can clearly see that, although in local regions, the output from certain classes appears to be denser, indicating that the conventional DFT extracts some common properties of the same class samples, on the whole, the features from different classes still tend to mix together. This shows that the conventional DFT pre-processing is not ideal for effectively extracting features from the raw radar signal. With the output from conventional DFT pre-processing, it could be quite challenging for the downstream classifier to produce accurate predictions.



(a) Conventional DFT (b) CubeLearn, Epoch = 1 (c) CubeLearn, Epoch = 4 (d) CubeLearn, Epoch = 15

Fig. 5. t-SNE visualization of the extracted features. Different colors represent different gestures.

On the other hand, we can see from Fig. 5(b)-(d), with the use of the CubeLearn module, the extracted features of different classes become more distinguishable as the training proceeds. This shows that during end-to-end training, the optimizer is able to adjust the weights in the CubeLearn module to produce more informative outputs. As the outputs of the pre-processing module become more distinguishable, it greatly reduces the burden of the downstream classifier, and makes the task of achieving a higher accuracy simpler. In later sections, we provide a further evaluation and discussion of the strengths and limitations of the proposed CubeLearn module.

VI. Experiment Setup

We conduct extensive experiments on three most commonly seen interaction tasks with mmWave radar, including hand gesture recognition, arm gesture recognition and human activity recognition. As there is no existing public dataset that contains raw complex radar data available for the above tasks, we collected our own dataset for evaluation. We try to follow existing publications on experiment settings and gesture/activity set designs. To understand the evaluation results, we provide the design of tasks, hardware settings, and software implementation details in this section.

A. Experiment Design

In this part we introduce our design of the hand gesture set, arm gesture set and activity set in our experiment.

1) Task 1: Hand Gesture Recognition (HGR): For hand gesture recognition, we follow the gesture set design of several previous works [3], [16], [20] and define our gesture set used in this work. Our gesture set design is shown in Fig. 6. The hand gesture set contains: (a) Pinch Index; (b) Pinch Pinky; (c) Slide Right; (d) Slide Left; (e) Rub Forward; (f) Rub Backward; (g) Rotate Left; (h) Rotate Right; (i) Fist; (j) Swipe Left; (k) Swipe Right; (l) Zoom Out. The palm is facing the sensor at the beginning of each gesture, and the gestures are

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

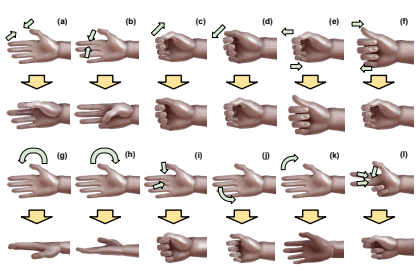


Fig. 6. Hand Gesture Set Design.

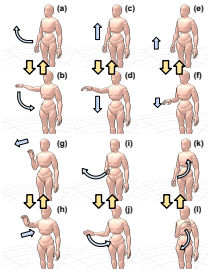


Fig. 7. Arm Gesture Set Design.

based on the right hand and are performed 20cm away in front of the sensor, as shown in Fig. 8(b).

Note that our hand gesture set design is much more difficult than the hand gesture sets in most of the previous works on hand gesture recognition with mmWave radar, as our gesture set only involves hand movement, mostly fingers, and does not include forearm movement.

2) Task 2: Arm Gesture Recognition (AGR): We design 12 commonly used gestures based on several previous works on radar arm gesture recognition [2], [36], as shown in Fig. 7. The gestures only involve right arm gestures and are performed 1.5m away in front of the radar, as shown in Fig. 8(c). The arm gesture set contains: (a) Side Lift; (b) Side Down; (c) Front Lift; (d) Front Down; (e) Forearm Lift; (f) Forearm Down; (g) Push; (h) Pull; (i) Swipe Right; (j) Swipe Left; (k) Diagonal Lift; (l) Diagonal Down.

3) Task 3: Human Activity Recognition (HAR): Another commonly seen application is human activity recognition (HAR). Different from HGR and AGR, HAR usually involves continuous movements, which means the activities do not have a certain 'start time' and 'end time'. Instead of trying to identify each start time as we do in gesture recognition, we simply cut the data into equal-length samples. We also follow the design in previous works [6], [7], [18], excluding those we believe that are not appropriate for participants to perform (e.g., crawling) or those that are too physically demanding (e.g., jumping) for typical smart-home applications. We include the following 6 activities in our dataset: (a) Marching on the spot; (b) Jogging on the spot; (c) Clapping; (d) Waving right hand; (e) Sweeping the floor with the right hand; (f) Rubbing left arm with right hand. The activities are performed 2m away in front of the radar in our data collection, as shown in Fig. 8(d).

B. Data Collection Setup

In this paper we use a commercial-off-the-shelf Texas Instruments IWR6843ISK mmWave sensor carried by MMWAVEICBOOST, and DCA1000EVM for raw data streaming. The sensor is mounted on a tripod, with a height of around 110cm, as shown in Fig. 8(a). Raw radar data is streamed and stored in PC continuously. For hand gesture and arm gesture recognition, the starting time of each action is logged and the data corresponding to each action is later extracted from the raw data file according to the starting time with a length of 1 second (10 CPIs).

We configured the radar with the following parameters: the starting frequency of each chirp is configured to 60.25 GHz with a frequency slope of 60 GHz/ms; the sampling rate is set to 10^7 samples per second and we take 256 ADC samples at each receiver antenna during a chirp; the period of each CPI is 100ms, containing 128 chirp loops where the 3 transmitters are activated one by one.

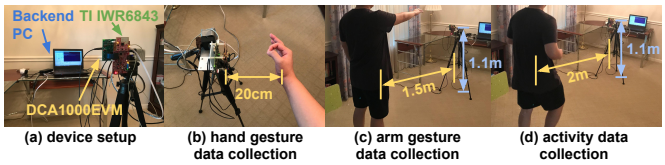


Fig. 8. Experiment Settings.

We invite 8 participants for data collection[3], including 4 males and 4 females, aged 26 to 55, with height approximately from 160cm to 175cm, and bodyweight approximately from 45kg to 90kg. For each participant, we collected 30 samples per hand/arm gesture, and 1 minute of activity, which is also cut into 30 samples. The background environment of data collection could vary across participants, but the experiment settings are the same. The data from the 8 participants are divided into two parts: 6 people are used as 'in-set' users, whose data is used for training and 'in-set' testing; the remaining 2 people are 'out-of-set' users, whose data is used to test the generalization abilities of the trained model. For 'in-set' users, 15 of the 30 samples of each gesture/activity is used for training, 5 used for validation and the remaining 10 used for testing, while for 'out-of-set' users, all 30 samples of each gesture/activity are used for testing.

C. Neural Network Implementation

We implement the neural networks with PyTorch. The complex layers are implemented based on an open-source Python module called CplxModule[4]. For Range and Doppler transformations, we configure the output size of the linear layer to be the same as the input size. We only use azimuth AoA information in this work, and for extracting AoA information, we configured the output size to 64 instead of the input size 8, so that the pre-processed data can be better fed into the convolutional layer for further feature extraction. For D-A-T and R-D-A-T pre-processing, we use the first half of the raw data on sample and chirp axes because of GPU memory limitation. This does not lose much information on Range and Doppler dimensions, as we will discuss in Section X.

For the implementation of the neural network classifiers, we use the same settings for each model. The kernel size for the convolutional layers are set to 3 and the channels are set to 4, 8, and 16, respectively. We apply max-pooling operation of size 2 on each dimension after each convolutional layer excluding the CPI dimension. The LSTM is set to have a hidden size of 512. The output size of the three fully-connected layers is set to 512, 128 and 12/6 (depending on the number of classes), respectively.

[3]The study has received ethical approval *CS_C1A_021_018*

[4]https://github.com/ivannz/cplxmodule

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

We use the cross-entropy loss to train the network. The neural networks are trained with Adam optimizer. Each model is trained for 30 epochs, and the best weights are saved according to validation accuracy and loss (accuracy is prioritized than loss).

VII. Evaluation

In this section we first evaluate the accuracy of DFT pre-processing based pipelines and CubeLearn based pipelines on HGR task, then extend to AGR and HAR tasks. Some of the pipelines with conventional DFT pre-processing are widely adopted by previous works on related topics, e.g., D-T pre-processing + CNN classifier [7], [15]–[18] and R-D-T pre-processing + CNN-LSTM classifier [25], [26], etc. As a result, they also serve as a comparison to previous related works.

We also evaluate the trained models on the 'out-of-set' users whose data is not included in the training. Generalization accuracy tends to be lower, as recognition accuracy can be impacted by the shape and size of the user's hand/body, how the user performs the action, as well as the environmental noise. This situation can certainly be mitigated by collecting more training data with various users and environments, or adopt transfer learning and life-long learning techniques. As our main aim here is to evaluate the generalization ability of our proposed CubeLearn based methods against conventional DFT pre-processing based pipelines, we do not focus on improving the generalization ability of a specific model/task here.

Besides validating the performance of our proposed CubeLearn module, this section also serves as a comparison between pre-processing and classifier combinations on different tasks.

A. Hand Gesture Recognition Evaluation

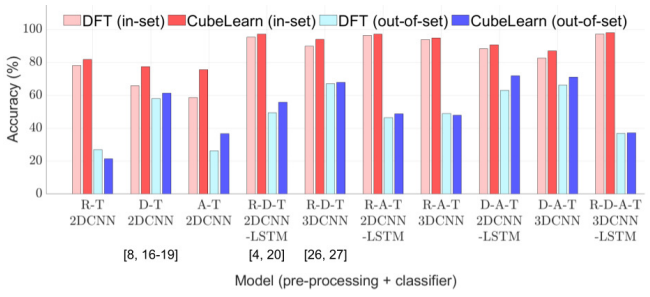


Fig. 9. Hand Gesture Recognition Accuracy.

We first evaluate our proposed CubeLearn module against conventional DFT on HGR task. The results are shown in Fig.9.

1) Impact of CubeLearn Module:

We can see from the figure that, with CubeLearn, the model is able to have consistently better or at least comparable performance than the DFT pre-processing counterparts. This is especially pronounced on simpler models, e.g., R-T pre-processing + 2DCNN classifier, A-T pre-processing + 2DCNN classifier and the widely adopted D-T pre-processing + 2DCNN classifier, with over 10% of accuracy improvement for the latter two models. Even for some of the models that already have a reasonably high recognition accuracy ($\geq 90\%$) with conventional DFT pre-processing, our proposed CubeLearn module is usually able to further improve attained accuracies by around $1 \sim 5\%$. These results show that our proposed CubeLearn module is a powerful tool to boost recognition accuracy on motion recognition tasks with mmWave radar.

2) Impact of the Combination of Pre-processing and Classifier: For 'in-set' testing, we generally have better performance with higher dimensional pre-processing. R-D-A-T pre-processing based model achieves the best testing accuracy of 97.36% with conventional DFT and 98.06% with CubeLearn. And also, two-dimensional pre-processing (R-D-T, R-A-T and D-A-T) have significantly better performance than one-dimensional pre-processing (R-T, D-T, A-T). Higher dimensional pre-processing inherently has more informative features, and the additional information can be utilized to better distinguish between different hand gestures. Also for R-D-T, R-A-T and D-A-T pre-processing, we find that 2DCNN-LSTM classifier behaves better than 3DCNN classifier, probably because the LSTM can better track the temporal relationships between frames in a gesture sample through the explicit latent feature space.

3) Generalization to Out-of-set Users:

When encountering out-of-set users, our proposed CubeLearn can still achieve better performances than DFT pre-processing for the majority of the models.

Having a closer investigation we can observe that the CubeLearn tends to further increase the generalization ability on more generalizable models, as these models capture more features that represents the underlying physical model, rather than some special properties of the training data. For example, in D-A-T + 2DCNN-LSTM classifier, with DFT pre-processing module the model is able to have around 63% of testing accuracy on 'out-of-set' subjects, and the proposed CubeLearn module can improve the accuracy to around 72%.

Comparing across different models, we find that the models with Doppler information have better generalization abilities. This is because Doppler information is invariant to the location where the gesture is performed, as long as the gesture or activity is oriented towards the radar. As CubeLearn can extract more distinguishable Doppler features through a learnable complex linear layer, the generalization abilities of Doppler-based models tend to further improve compared to conventional DFT pre-processing counterparts.

However, with the strong feature extracting ability of the CubeLearn module, some specific characteristics of the training samples could also be extracted and carried over, which sometimes makes it harder for the model to generalize to previous unseen subjects. For example for R-T pre-processing + 2DCNN classifier, although CubeLearn can attain better test performance with 'in-set' users, it generalizes less well than conventional DFT pre-processing, as the Range feature itself is inherently not able to generalize well on 'out-of-set' subjects, as the conventional DFT based pre-processing model achieving less than 30% accuracy. Luckily when solving real-world problems, we would most likely pick models that use

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

features with strong generalization ability, and in these cases, CubeLearn module could also be beneficial to the testing accuracy on 'out-of-set' users.

With respect to radar data cube slicing, differently from 'in-set' testing, higher dimensional pre-processing does not always lead to better generalization performance. For example, the D-A-T + 3DCNN model has better generalization ability than R-D-A-T + 3DCNN-LSTM model, also because that the R-D-A-T features overfit to the training subjects. This could also be a function of the training set size - a dataset of hundreds of users would likely generalize well, even for higher dimensional models.

B. Generalization to AGR and HAR Task

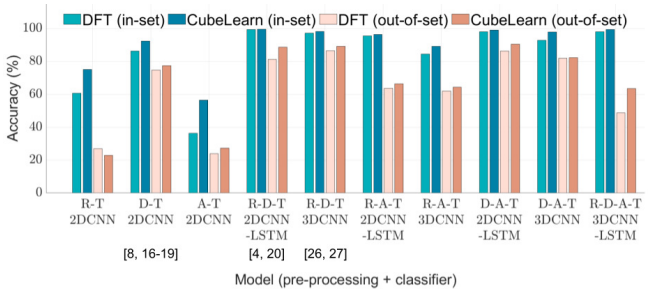


Fig. 10. Arm Gesture Recognition Accuracy.

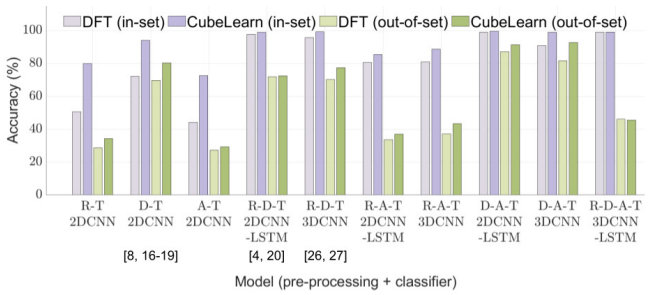


Fig. 11. Human Activity Recognition Accuracy.

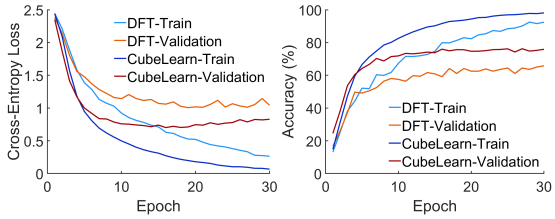
We further evaluate our proposed method against conventional DFT on the other two tasks, and the results are shown in Fig. 10 and Fig. 11. AGR task is similar to HGR task, where each gesture is represented by a sequence of frames, but it is much easier as the movements are bigger which makes them more obvious to mmWave radar. HAR is fundamentally different as it mainly involves recognizing continuous and repetitive movements.

From the 'in-set' testing result we can see that, the model performances on AGR and HAR are generally better than HGR task. Still, our CubeLearn module can consistently improve the performance of different pipelines, by extracting the most informative features from the raw radar signal cube. The proposed module can be extremely beneficial sometimes, e.g., increasing the accuracies of some of the simpler models (R-T, D-T, A-T) by almost 15% to 20%. This is an important finding, as R-T and D-T are the most basic signals which all FMCW radars will support, regardless of the number of antennas, and they are more computational efficient for edge devices. This means that a simpler radar (with respect to hardware configuration) could achieve the performance of a much more sophisticated device, merely through the use of CubeLearn.

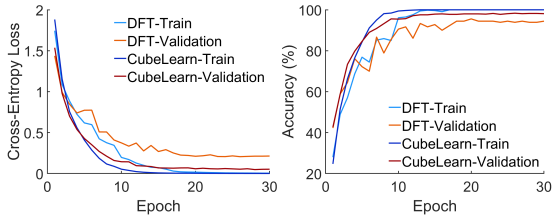
Different from HGR/AGR case, in HAR task we find that using LSTM in the downstream classifier does not always yield better performance. For example when using R-A-T pre-processing, the 3DCNN classifier has better performance than the 2DCNN-LSTM classifier. This suggests that the LSTM might be better at recognizing gestures than continuous motion, and for continuous motion recognition, both classifiers are worth trying.

The accuracies for 'out-of-set' test are also higher as the tasks themselves are less challenging to generalize due to their more pronounced motion. The accuracies of some models on 'out-of-set' subjects are comparable to the accuracies on 'in-set' subjects, such D-T + 2DCNN in HAR task. By analyzing the 'out-of-set' test results, we can arrive at a similar conclusion as in the HGR task, i.e., our proposed CubeLearn module can be beneficial for most models, and Doppler features are more important for motion recognition than Range or AoA features.

C. Convergence Analysis



(a) HGR: D-T pre-processing + 2DCNN classifier



(b) HAR: R-D-T pre-processing + 3DCNN classifier

Fig. 12. Visualization of the training processes.

We further compare the convergence speed of methods based on DFT pre-processing and CubeLearn. We visualize the training process, including the training accuracy/loss and the validation accuracy/loss, for D-T pre-processing + 2DCNN classifier on HGR task and R-D-T pre-processing + 3DCNN classifier on HAR task in Fig. 12. In both cases, the model with CubeLearn is able to converge in fewer epochs, typically within 15 epochs, with loss decreasing and accuracy increasing much faster. Furthermore, from Fig. 12 (b) we can see that, though the models converge to approximately the same training loss and accuracy, completely fitting the training set, there is a constant validation loss/accuracy difference between the models, which shows the network with the CubeLearn module can actually better learn the physical model behind the task. Note that this shows that the superior performance of the CubeLearn module is not merely because of more trainable parameters, as in this case CubeLearn based model

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

and conventional DFT based model both completely fit the training data, while the CubeLearn based model still yields better performance on the validation/testing set.

VIII. Ablation Study

A. Impact of Complex Layer Learning Rate

The learning rate of the CubeLearn module controls the speed at which the complex layer weights vary, which can be set differently from the learning rate of the neural network classifier. With a higher CubeLearn module learning rate, the weights of the complex linear layers change faster during training - the coupling between the pre-processing and classification networks could cause the system as a whole to converge to a sub-optimal set of weights. To find the optimal learning rate for the complex layers (or the ratio between the learning rates of the CubeLearn module and the classifier), in this part we study the impact of the CubeLearn module learning rate on the model overall accuracy. We use the D-T pre-processing + 2DCNN classifier model on HGR task here for demonstration. The learning rate of the neural network classifier is set to 0.0003 for all the cases. Each model is run multiple times. The result is shown in Fig. 13.

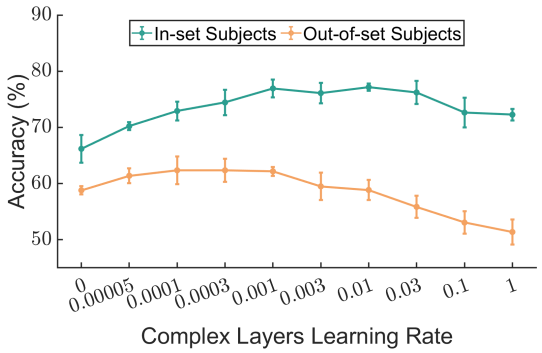


Fig. 13. Test accuracy V.S. different learning rates.

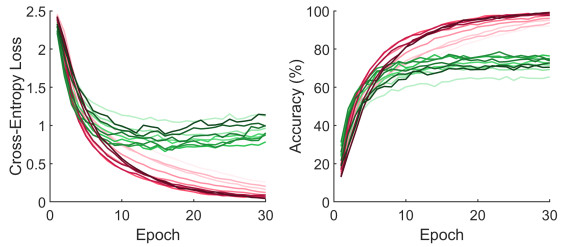


Fig. 14. Loss and Accuracy curve. Red: Training; Green: Validation. Darker color represents higher CubeLearn module learning rate.

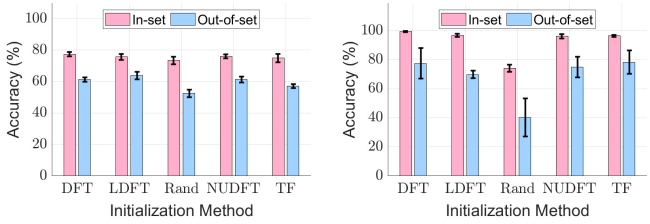
From the results we can see that, both 'in-set' and 'out-of-set' testing, with the increase of the CubeLearn module learning rate, the model accuracy first increases, then decreases. Note that when the learning rate is 0, the model degenerates to a conventional fixed DFT pre-processing + 2DCNN classifier model. With the learning rate from 0.00005 to 0.0001, the optimizer is able to adjust the weights of the CubeLearn module to achieve better model accuracy, for 'in-set' and 'out-of-set' subjects test. With the learning rate further increasing, the model is likely to overfit to the training subjects, and even to the training samples. As shown in the figure, the generalization accuracy begins to decrease when the learning rate of the CubeLearn module is larger than 0.001, which shows that the model is overfitted to the training subjects. The 'in-set' test accuracy is able to be maintained at a relatively high level until the learning rate reaches 0.03. When the learning rate is larger than 0.03, the model further overfits to the training samples, leading to an accuracy degradation on the 'in-set' test as well.

Fig. 14 shows the training and validation loss and accuracy curve of the models with different CubeLearn module learning rates. We can see that for training set, with a higher learning rate the model is able to converge at a smaller training loss, and larger training accuracy. However, a suitable learning rate could make the model converge faster. For validation set, either a learning rate that is too small or too large yields sub-optimal performance, and the smallest loss and highest accuracy can only be achieved with a proper learning rate of the complex layers, which is 0.001 in this case. Note that the optimal learning rate ratio between the CubeLearn module and the neural network classifier can vary for different pre-processing and classifier combinations as well as on different tasks.

B. Impact of Weight Initialization Method

The initialization of the weights in the stacked complex linear layers affects the model accuracy as well as convergence. In this part we try different weight initialization strategies, including: (1) Log-DFT initialization, where the weights are initialized with Fourier Transform bases more focusing on the low frequency part (**LDFT**); (2) Default Random initialization (He Initialization [37]) for both real and imaginary parts of the complex weights (**Rand**); (3) Random Non-uniform Discrete Fourier Transform bases initialization, where the initialize weights are based on random Fourier Transform bases (sorted) (**NUDFT**); (4) Target Focus initialization, where the information (e.g., distance, velocity, AoA) regarding the target is known and the bases of the Fourier initialization are generated close to the target frequency with normal distribution (**TF**).

We compare the complex linear layers initialization methods with two most common models: D-T pre-processing + 2DCNN classifier on HGR task and R-D-T pre-processing + 3DCNN classifier on HAR task. The evaluation result is shown in Fig. 15, where we can see that the model generally is not very sensitive to the weight initialization method, and is able to converge in each case.



(a) D-T + 2DCNN classifier (b) R-D-T + 3DCNN classifier

Fig. 15. Initialization Method Ablation Study.

Log-DFT initialization focuses on the low frequency part, which produces slightly better generalization accuracy for the

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

HGR task, because in the Range domain, the high frequencies correspond to distant targets, which is the background environment in this case. However, we observe a slight performance downgrade for the HAR task, because the user is 2 meters away from the sensor in the HAR task, which is approximately in the middle of the frequency spectrum rather than low-frequency part. With the Log-DFT initialization focusing on the low-frequency part, there are in fact fewer bases initially focus on the target. As a result, Log-DFT might be beneficial for recognizing targets that are close to the sensor, however, this type of initialization needs careful design.

We find that with random initialization, the model has worse 'in-set' and 'out-of-set' accuracy. Especially for the HAR case, the accuracy drops to below 80% and the 'out-of-set' generalization accuracy even drops to below 40%. For HGR the 'out-of-set' generalization accuracy also drops significantly. Using random initialization basically means giving up the prior knowledge about radar signal processing, which is not desirable, especially when the number of samples in the training set is limited.

The Random NDUFT initialization is theoretically quite similar to DFT bases initialization, for the bases are sampled with uniform distribution across the frequency spectrum and re-organized from low frequency to high frequency. However, from the result we see that this kind of initialization is not as stable as uniform DFT bases initialization, with slightly worse average performance.

Target focus initialization is very hard to use in practice, since it requires prior knowledge of the location, velocity and AoA of the target, and a lot of engineering effort. Interestingly, we do not observe better performance even with careful parameter tuning.

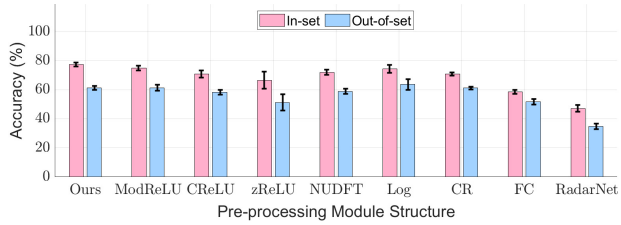
In conclusion, in practice it is reasonable to simply use DFT bases to initialize the weights of the complex linear layers due to their simplicity, high accuracy and stable performance.

C. Impact of Network Structure

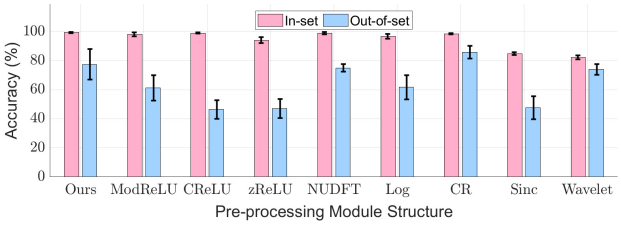
We further conduct an ablation study of the pre-processing module structure. We also use the D-T pre-processing + 2DCNN classifier model on HGR task and R-D-T pre-processing + 3DCNN classifier model on HAR task here, as they are the most commonly seen models in previous works. Besides, learnable D-T pre-processing can be directly compared to RadarNet [12] and learnable R-D-T pre-processing can be directly compared to 2D Sinc/Wavelet Filters [11]. As mentioned in Section II, RadarNet and 2D Sinc/Wavelet Filters are two previously proposed methods for end-to-end radar signal processing.

We compare the performance of our proposed structure to the following variations: (1) Adding Activation between Complex Linear Layers (**ModReLU** [38], **ℂReLU** [39], **zReLU** [40]); (2) Learnable Non-uniform Discrete Fourier Transform (**NUDFT**), where we constrain the learned weights to still represent Fourier Transform bases; (3) Log after Modulus (**Log**), where the log operation is applied after taking the modulus to represent power spectrum as in [12], [41]; (4) Clutter Removal (**CR**), where we subtract the chirp average from the raw radar data cube before feeding it into the network; (5) Fully-Complex Network (**FC**), where we do not take modulus after pre-processing, and use a fully complex version of the downstream classifier; (6) **RadarNet** [12]; and (7) 2D **Sinc** Filters and 2D **Wavelet** Filters [11].

As RadarNet [12] is designed for CW radar rather than FMCW radar, we first do Range DFT on the raw radar data, and then apply the pre-processing module introduced in RadarNet for Doppler estimation on each Range bin, then sum along the Range axis. Besides, we use log operation instead of tanh which is suggested in the original paper, as the network does not converge when using tanh in our case. For Sinc and Wavelet Filters [11], we apply it on each CPI and use the CPI number as an additional dimension, since the radar configuration is different in this work. Besides, we find that the hyper-parameters greatly impact the Sinc/Wavelet Filter performance, so we try different combinations and select the best one. Both RadarNet and Sinc/Wavelet Filters takes real-valued data, and we disregard the imaginary part of the data.



(a) HGR: D-T + 2DCNN classifier



(b) HAR: R-D-T + 3DCNN classifier

Fig. 16. Structure Ablation Study.

The result is shown in Fig. 16. We can see that, adding an activation function between the complex linear layers decreases the model performance, especially the generalization ability. This is probably because in the stacked complex linear layers, extracting the Doppler information through the transformation of the latter complex linear layer relies on the output phase information of the previous complex linear layer. Adding a non-linear activation between the complex linear layers, especially ℂReLU and zReLU, could result in the valuable phase information being altered or lost.

NUDFT structure produces a slightly worse result than CubeLearn, since the change of the weights is restricted and not as flexible. As for adding the log operation after the CubeLearn module, we do not observe performance improvement. In fact, for the HAR case, adding the log operation downgrades the generalization ability, probably because the difference of peak magnitude and average magnitude is smaller, making the learned features less distinguishable.

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

Clutter removal operation removes reflection of static targets (e.g., background), and make the model learn properties of the moving target, which increases the model generalization ability in HAR case. On the other hand, as it actually cancels out all the static information, including the information of the target, for HGR task where the hand is very close to the sensor and occupying a larger field of view, removing the static information slightly decreases the model performance. Besides, if we are to recognize the static target, e.g., in static gesture recognition, clutter removal will certainly not work. As a result, whether adding clutter removal is beneficial or not depends on the task. For tasks to recognize moving targets that are relatively further away, adding clutter removal could increase the model generalization ability by removing the background reflections. While recognizing partial static or static targets, especially in cases where the target occupies a large field of view and the reflection from the environment has limited impact, adding clutter removal could possibly decrease the performance.

Using the processing module in RadarNet lowers the accuracy. This is because RadarNet only consumes the real input, so it loses half of the information, which leads to a worse accuracy. 2D Sinc Filters and 2D Wavelet Filters also takes real-valued input, so the accuracy is not as good as other structures. We find that compared to our proposed CubeLearn with complex linear layers, 2D Sinc Filters and 2D Wavelet Filters are very easy to become overfitted to the training samples, and are very sensitive to the layer parameters. For 'out-of-set' test, we observe that 2D Wavelet Filters have competent generalization abilities, much higher than 2D Sinc Filters.

In conclusion, compared to other variations, our proposed stacked complex linear layers achieve good and stable performance. Adding clutter removal could possibly improve the generalization abilities of the model in some tasks, but needs to be used with caution.

IX. Running Time Analysis

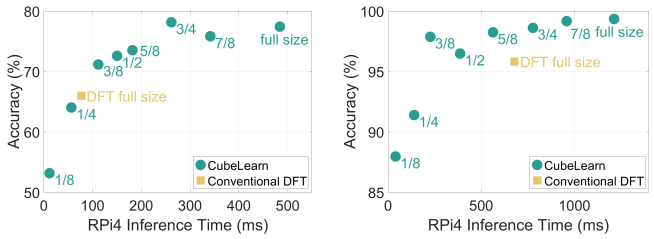
The Discrete Fourier Transform is usually implemented with Fast Fourier Transform algorithm, which is very computationally efficient ($\mathcal{O}(n \log n)$). Using the stacked complex linear layers introduces more parameters in the network, which will result in larger complexity and longer inference time. In this part we analyze the running time of the proposed method against classical hybrid baselines on PC and Raspberry Pi. The experiment PC we use features an AMD Ryzen 3800X CPU with 64GB memory, and Nvidia RTX2080Ti graphics card, and the Raspberry Pi we use is Raspberry Pi 4 with 8GB main memory. For DFT based pre-processing, we use 'torch.fft' in 'pytorch' package for possible GPU acceleration when testing on PC, and use 'scipy.fftpack.fft' in 'scipy' package on Raspberry Pi since 'torch.fft' is not supported on ARM architecture. The structures of the neural networks are discussed in Sec. VI-C. Note that the running time reported in this section is based on models for HGR/AGR task.

The result is summarized in TABLE. III. Not surprisingly, the computation of complex linear layers is more expensive than FFT, as each layer is actually called multiple times during inference. With GPU acceleration on PC, adding the learnable complex layers would add 47.7% of inference time for one sample on average (SD=15%). When running with the ARM CPU on Raspberry Pi 4, the running time for each sample increases 152.7% on average (SD=163.1%). The computational cost grows exponentially with the number of dimensions of the raw data cube used. For example, with D-A-T or R-D-A-T pre-processing, the neural network has to perform complex linear transformation on all three dimensions, which leads to a large increase in inference time. However, in practice, complex models (e.g., D-A-T and R-D-A-T pre-processing based models) are less likely to be used on edge devices. For simpler models that can run real time on edge devices, CubeLearn could greatly improve the recognition accuracy, as shown in previous sections, with acceptable complexity increase. Furthermore, we can have a smaller input size with our proposed end-to-end network, to achieve better performance and even smaller running time than DFT baselines, as introduced in the next section.

Model	PC Time (ms)		Raspberry Pi Time (ms)	
	DFT	CubeLearn	DFT	CubeLearn
R-T + 2DCNN	0.99	1.25	15.00	35.71
D-T + 2DCNN	1.05	1.65	77.41	484.13
A-T + 2DCNN	1.08	1.61	39.51	165.32
R-D-T + 2DCNN-LSTM	1.63	2.27	625.72	1018.70
R-D-T + 3DCNN	1.14	1.75	419.31	745.10
R-A-T + 2DCNN-LSTM	1.62	2.21	308.38	528.02
R-A-T + 3DCNN	1.15	1.69	222.17	357.00
D-A-T + 2DCNN-LSTM	3.18	5.29	1486.26	2456.30
D-A-T + 3DCNN	2.92	5.02	1626.13	2513.14
R-D-A-T + 3DCNN-LSTM	18.73	24.38	-	-

TABLE III
RUNNING TIME ON PC AND RASPBERRY PI PER SAMPLE.

X. Impact of Raw Data Size



(a) HGR: D-T + 2DCNN classifier (b) HAR: R-D-T + 3DCNN classifier

Fig. 17. Accuracy and inference time with different size of raw data input.

The raw radar signal contains repetitive patterns on ADC samples in a chirp, and across chirps in a CPI. As a result, lowering the number of samples and chirp number in a CPI would possibly still yield a similar level of recognition performance, while making the neural network much more light-weight. In this part we tested the performance of two models with various input sizes: D-T pre-processing + 2DCNN classifier on HGR task and R-D-T pre-processing + 3DCNN classifier on HAR task, as well as the running speed on Raspberry Pi4. Note that the running time of the R-D-T pre-processing + 3DCNN classifier is different from the one reported in Section IX

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

because each sample lasts 2 seconds for HAR task, instead of 1 second for HGR task which we studied previously.

The input here is actually a series of sample-chirp data slices. For both axes, we try using the first 1/8, 1/4, ..., 7/8 of the data points on each dimension as input. As we have a smaller size of the input, the size of the pre-processing module and the neural network classifier is also smaller, which reduces the computational complexity. The result is shown in Fig. 17, the text beside each data point refers the ratio of the samples and chirps we use for each CPI. To have a fair comparison, each model is run multiple times.

We can see that the full-sized input actually contains redundant information. With even 3/8 of the original input size on sample and chirp dimension, we are still able to achieve relatively high performance, better than full sized input with conventional DFT pre-processing. For the HAR case, with 3/8, 1/2 or 5/8 size on each dimension of the original input, the inference time on Raspberry Pi 4 of the CubeLearn based model is smaller than conventional DFT pre-processing based model with full-sized input, but achieves better classification accuracy. Especially with 3/8 of size on both dimensions, the accuracy could reach 98% while the inference time is approximately 200ms.

In fact, for specific tasks, we can further reduce the complexity and running time by, e.g., using fewer neurons to represent the transformation output with the initialization of only the Fourier bases corresponding to the distance and velocity of the target. Besides, we can also utilize hardware-level parallel, processing the data of the current CPI while receiving the data for the next CPI. As a result, we believe that with an optimized implementation as future work, our proposed end-to-end model would execute in real-time on resource constrained devices, with superior performance to the DFT front-end.

XI. Discussion

A. Model Selection

As we have discussed in Section VII, R-D-A-T pre-processing + 3DCNN-LSTM classifier shows good accuracy on different motion recognition tasks, especially with our proposed learnable pre-processing module, as information from all three dimensions (Range, Doppler and AoA) are utilized for target classification. However, it mainly has two problems which limit its adoption. The first one is computational complexity, which can be higher especially on edge devices without GPU acceleration (an issue similarly shared by D-A-T pre-processing). Another problem is that it requires multiple receivers for AoA estimation, which is not always supported by the hardware itself, so this type of model is best suitable for MIMO radar with abundant computational resource.

Doppler information is the most important for motion recognition. As a result, considering the complexity and accuracy on both 'in-set' users and 'out-of-set' users, D-T or R-D-T pre-processing can be good choices, especially for 1Tx-1Rx radar configuration. Besides, D-T pre-processing can be extended to other types of radar as well, e.g., Continuous-wave (CW) radar.

B. Robustness

For learning based methods, including our proposed neural network in this paper, the generalization ability, i.e., robustness to different situations, is always an important concern. In this paper, we make a preliminary investigation into this topic by collecting data from different users and in different environments. We suggest that more comprehensive research on this topic can be done in future works.

C. Utilizing Elevation Angle Information

In this paper, we evaluated and compared models that are based on Range, Doppler and Azimuth Angle information as inputs. With a 2D MIMO attena array, the radar is also able to capture elevation angle information, which can potentially be used for human gesture/activity recognition as well. However, in this case, the network design could be different. If we try to utilize all the information simultaneously, i.e., range, Doppler, azimuth angle and elevation angle, then we would need to apply a 4D convolutional layer which is not commonly seen in neural network architectures. Alternatively, we can explore other network structures.

D. Other Structures

While the order of the linear layers is worth studying, other structures for extracting information from raw radar signals are also worth exploring. Transformer architecture [42] was proposed in recent years, which has shown very good performance on a wide variety of tasks like machine translation. With positional encoding for different virtual antennas, we could use the encoder part of the transformer model, potentially a complex-weighted version, to extract Range, Doppler and AoA features together into a latent embedding and directly perform classification on the latent vector. In this way, we can get rid of the downstream CNN/CNN-LSTM architecture as well. We believe this is also a promising direction to study in future work.

XII. Conclusion

In this paper, we propose CubeLearn, a learnable pre-processing module to replace conventional Discrete Fourier Transform (DFT) pre-processing in mmWave FMCW radar gesture/activity recognition pipelines. We demonstrate that through end-to-end training, our proposed CubeLearn module is able to extract more distinguishable features than conventional DFT pre-processing, which makes it easier for the downstream classifier to make correct predictions and improve the overall model accuracy. We evaluate our proposed method on our own collected dataset of hand gestures, arm gestures and activities. Results show that for all the tasks and different pre-processing/classifier combinations, the classification accuracy can be consistently improved with the proposed CubeLearn module, and the training is able to converge in fewer epochs. We envision our proposed method as a small step towards a universal feature extractor for end-to-end deep learning on raw mmWave radar data. Future works would be focusing on exploring more efficient end-to-end structures and improving the model robustness.

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2023.3237494

REFERENCES

[1] A. Singh, S. U. Rehman, S. Yongchareon, and P. H. J. Chong, "Multi-resident non-contact vital sign monitoring using radar: A review," *IEEE Sensors Journal*, 2020.

[2] S. Palipana, D. Salami, L. A. Leiva, and S. Sigg, "Pantomime: Mid-air gesture recognition with sparse millimeter-wave radar point clouds," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 5, no. 1, pp. 1–27, 2021.

[3] S. Wang, J. Song, J. Lien, I. Poupyrev, and O. Hilliges, "Interacting with soli: Exploring fine-grained dynamic gesture recognition in the radio-frequency spectrum," in *Proceedings of the 29th Annual Symposium on User Interface Software and Technology*, 2016, pp. 851–860.

[4] F. Jin, A. Sengupta, and S. Cao, "mmfall: Fall detection using 4-d mmwave radar and a hybrid variational rnn autoencoder," *IEEE Transactions on Automation Science and Engineering*, 2020.

[5] B. Wang, L. Guo, H. Zhang, and Y.-X. Guo, "A millimetre-wave radar-based fall detection method using line kernel convolutional neural network," *IEEE Sensors Journal*, vol. 20, no. 22, pp. 13 364–13 370, 2020.

[6] A. D. Singh, S. S. Sandha, L. Garcia, and M. Srivastava, "Radhar: Human activity recognition from point clouds generated through a millimeter-wave radar," in *Proceedings of the 3rd ACM Workshop on Millimeter-wave Networks and Sensing Systems*, 2019, pp. 51–56.

[7] K. Ahuja, Y. Jiang, M. Goel, and C. Harrison, "Vid2doppler: Synthesizing doppler radar data from videos for training privacy-preserving activity recognition," in *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, 2021, pp. 1–10.

[8] A. Khamis, B. Kusy, C. T. Chou, M.-L. McLaws, and W. Hu, "Rfwash: a weakly supervised tracking of hand hygiene technique," in *Proceedings of the 18th Conference on Embedded Networked Sensor Systems*, 2020, pp. 572–584.

[9] S. Ahmed, K. D. Kallu, S. Ahmed, and S. H. Cho, "Hand gestures recognition using radar sensors for human-computer-interaction: A review," *Remote Sensing*, vol. 13, no. 3, p. 527, 2021.

[10] X. Li, Y. He, and X. Jing, "A survey of deep learning-based human activity recognition in radar," *Remote Sensing*, vol. 11, no. 9, p. 1068, 2019.

[11] T. Stadelmayer and A. Santra, "Data-driven radar processing using a parametric convolutional neural network for human activity classification," 2021.

[12] W. Ye, H. Chen, and B. Li, "Using an end-to-end convolutional network on radar signal for human activity classification," *IEEE Sensors Journal*, vol. 19, no. 24, pp. 12 244–12 252, 2019.

[13] R. Zhao, X. Ma, X. Liu, and J. Liu, "An end-to-end network for continuous human motion recognition via radar radios," *IEEE Sensors Journal*, 2020.

[14] X. Zheng, Z. Yang, K. He, and H. Liu, "Hand gesture recognition based on range doppler-angle trajectory and lstm network using an mimo radar," in *Eleventh International Conference on Signal Processing Systems*, vol. 11384. International Society for Optics and Photonics, 2019, p. 113840P.

[15] B. Dekker, S. Jacobs, A. Kossen, M. Kruithof, A. Huizing, and M. Geurts, "Gesture recognition with a low power fmcw radar and a deep convolutional neural network," in *2017 European Radar Conference (EURAD)*. IEEE, 2017, pp. 163–166.

[16] Q. Wu, D. Zhao *et al.*, "Dynamic hand gesture recognition using fmcw radar sensor for driving assistance," in *2018 10th International Conference on Wireless Communications and Signal Processing (WCSP)*. IEEE, 2018, pp. 1–6.

[17] W. Jiang, Y. Ren, Y. Liu, Z. Wang, and X. Wang, "Recognition of dynamic hand gesture based on mm-wave fmcw radar micro-doppler signatures," in *ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2021, pp. 4905–4909.

[18] R. Zhang and S. Cao, "Real-time human motion behavior detection via cnn using mmwave radar," *IEEE Sensors Letters*, vol. 3, no. 2, pp. 1–4, 2018.

[19] Y. Wang, S. Wang, M. Zhou, Q. Jiang, and Z. Tian, "Ts-i3d based hand gesture recognition method with radar sensor," *IEEE Access*, vol. 7, pp. 22 902–22 913, 2019.

[20] M. Scherer, M. Magno, J. Erb, P. Mayer, M. Eggimann, and L. Benini, "Tinyradarnn: Combining spatial and temporal convolutional neural networks for embedded gesture recognition with short range radars," *IEEE Internet of Things Journal*, pp. 1–1, 2021.

[21] P. Goswami, S. Rao, S. Bharadwaj, and A. Nguyen, "Real-time multi-gesture recognition using 77 ghz fmcw mimo single chip radar," in *2019 IEEE International Conference on Consumer Electronics (ICCE)*. IEEE, 2019, pp. 1–4.

[22] J.-T. Yu, L. Yen, and P.-H. Tseng, "mmwave radar-based hand gesture recognition using range-angle image," in *2020 IEEE 91st Vehicular Technology Conference (VTC2020-Spring)*. IEEE, 2020, pp. 1–5.

[23] R. Huang, Z. Li, S. Wang, R. Wang, J. Li, and Z. Xu, "A rd-t network for hand gesture recognition based on millimeter-wave sensor," in *2020 IEEE 5th International Conference on Signal and Image Processing (ICSIP)*. IEEE, 2020, pp. 308–312.

[24] Y. Wang, Z. Zhao, M. Zhou, and J. Wu, "Two dimensional parameters based hand gesture recognition algorithm for fmcw radar systems," in *International Conference on Wireless and Satellite Systems*. Springer, 2019, pp. 226–234.

[25] P.-h. CHEN, Y.-j. BAI, and W. Jun, "Gesture recognition using lfmcw radar and convolutional neural network," *DEStech Transactions on Computer Science and Engineering*, no. cscme, 2019.

[26] S. Hazra and A. Santra, "Short-range radar-based gesture recognition system using 3d cnn with triplet loss," *IEEE Access*, vol. 7, pp. 125 623–125 633, 2019.

[27] S. Hazra and A. Santra, "Radar gesture recognition system in presence of interference using self-attention neural network," in *2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA)*. IEEE, 2019, pp. 1409–1414.

[28] T. Sakamoto, X. Gao, E. Yavari, A. Rahman, O. Boric-Lubecke, and V. M. Lubecke, "Hand gesture recognition using a radar echo i–q plot and a convolutional neural network," *IEEE sensors letters*, vol. 2, no. 3, pp. 1–4, 2018.

[29] X. Yao, X. Shi, and F. Zhou, "Human activities classification based on complex-value convolutional neural network," *IEEE Sensors Journal*, vol. 20, no. 13, pp. 7169–7180, 2020.

[30] W. Ye and H. Chen, "Human activity classification based on micro-doppler signatures by multiscale and multitask fourier convolutional neural network," *IEEE Sensors Journal*, vol. 20, no. 10, pp. 5473–5479, 2020.

[31] K. Rambach and B. Yang, "Mimo radar: time division multiplexing vs. code division multiplexing," 2017.

[32] R. Schmidt, "Multiple emitter location and signal parameter estimation," *IEEE transactions on antennas and propagation*, vol. 34, no. 3, pp. 276–280, 1986.

[33] M. Schutz, C. Decroze, M. Lalande, and B. Lenoir, "Neural networks to increase range resolution of fmcw radar," *IEEE Sensors Letters*, vol. 4, no. 8, pp. 1–4, 2020.

[34] M. Gall, M. Gardill, T. Horn, and J. Fuchs, "Spectrum-based single-snapshot super-resolution direction-of-arrival estimation using deep learning," in *2020 German Microwave Conference (GeMiC)*. IEEE, 2020, pp. 184–187.

[35] L. Van der Maaten and G. Hinton, "Visualizing data using t-sne." *Journal of machine learning research*, vol. 9, no. 11, 2008.

[36] H. Liu, Y. Wang, A. Zhou, H. He, W. Wang, K. Wang, P. Pan, Y. Lu, L. Liu, and H. Ma, "Real-time arm gesture recognition in smart home scenarios via millimeter wave sensing," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 4, no. 4, pp. 1–28, 2020.

[37] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1026–1034.

[38] M. Arjovsky, A. Shah, and Y. Bengio, "Unitary evolution recurrent neural networks," in *International Conference on Machine Learning*. PMLR, 2016, pp. 1120–1128.

[39] C. Trabelsi, O. Bilaniuk, Y. Zhang, D. Serdyuk, S. Subramanian, J. F. Santos, S. Mehri, N. Rostamzadeh, Y. Bengio, and C. J. Pal, "Deep complex networks," *arXiv preprint arXiv:1705.09792*, 2017.

[40] N. Guberman, "On complex valued convolutional neural networks," *arXiv preprint arXiv:1602.09046*, 2016.

[41] D. A. Brooks, O. Schwander, F. Barbaresco, J.-Y. Schneider, and M. Cord, "Complex-valued neural networks for fully-temporal micro-doppler classification," in *2019 20th International Radar Symposium (IRS)*. IEEE, 2019, pp. 1–10.

[42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *arXiv preprint arXiv:1706.03762*, 2017.

© 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Edinburgh. Downloaded on March 04,2023 at 09:27:25 UTC from IEEE Xplore. Restrictions apply.